```
# Step 1: Import Necessary Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, accuracy_score

# Step 2: Create Dataset Manually (SUV Data)
data = pd.DataFrame({
    'Age': [19, 35, 26, 27, 19, 27, 32, 25, 35, 26, 30, 40, 50, 48, 33],
    'EstimatedSalary': [19000, 20000, 43000, 57000, 76000, 58000, 150000, 33000, 65000, 80000, 87000, 120000, 90000
    'Purchased': [0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1]
})

print("Dataset:\n", data)

# Step 3: Prepare Data
X = data[['Age', 'EstimatedSalary']].values
y = data['Purchased'].values

# Step 4: Split Data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Step 5: Feature Scaling
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# --------------------------------------------------
# Step 6: Logistic Regression (In-built)
# --------------------------------------------------
from sklearn.linear_model import LogisticRegression

classifier = LogisticRegression()
classifier.fit(X_train, y_train)

# Step 7: Predictions
y_pred = classifier.predict(X_test)
y_train_pred = classifier.predict(X_train)

# Step 8: Evaluation
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)

print("\n--- In-built Model ---")
print("Confusion Matrix:\n", cm)
print("Accuracy:", accuracy)

# --------------------------------------------------
# USER DEFINED LOGISTIC REGRESSION
# --------------------------------------------------

# Sigmoid Function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Standardization function
def standardize(X):
    for i in range(X.shape[1]):
        X[:, i] = (X[:, i] - np.mean(X[:, i])) / np.std(X[:, i])
    return X

# F1 Score
def F1_score(y, y_hat):
    tp = tn = fp = fn = 0
    for i in range(len(y)):
```

```
        if y[i] == 1 and y_hat[i] == 1:
            tp += 1
        elif y[i] == 1 and y_hat[i] == 0:
            fn += 1
        elif y[i] == 0 and y_hat[i] == 1:
            fp += 1
        elif y[i] == 0 and y_hat[i] == 0:
            tn += 1

    precision = tp / (tp + fp + 1e-10)
    recall = tp / (tp + fn + 1e-10)

    return 2 * precision * recall / (precision + recall + 1e-10)

# Logistic Regression Class
class MyLogisticRegression:

    def initialize(self, X):
        weights = np.zeros((X.shape[1] + 1, 1))
        X = np.c_[np.ones((X.shape[0], 1)), X]
        return weights, X

    def fit(self, X, y, alpha=0.01, iterations=1000):
        weights, X = self.initialize(X)
        y = y.reshape(-1, 1)

        for i in range(iterations):
            z = np.dot(X, weights)
            h = sigmoid(z)
            gradient = np.dot(X.T, (h - y)) / len(y)
            weights -= alpha * gradient

        self.weights = weights

    def predict(self, X):
        X = np.c_[np.ones((X.shape[0], 1)), X]
        z = np.dot(X, self.weights)
        probs = sigmoid(z)

        return [1 if i > 0.5 else 0 for i in probs]


# Standardize again for custom model
X_train_std = standardize(X_train.copy())
X_test_std = standardize(X_test.copy())

# Train custom model
model = MyLogisticRegression()
model.fit(X_train_std, y_train)

# Predictions
y_pred_custom = model.predict(X_test_std)
y_train_custom = model.predict(X_train_std)

# F1 Scores
f1_train = F1_score(y_train, y_train_custom)
f1_test = F1_score(y_test, y_pred_custom)

print("\n--- User Defined Model ---")
print("F1 Score (Train):", f1_train)
print("F1 Score (Test):", f1_test)

# Confusion Matrix
cm_custom = confusion_matrix(y_test, y_pred_custom)
accuracy_custom = (cm_custom[0][0] + cm_custom[1][1]) / np.sum(cm_custom)

print("Confusion Matrix:\n", cm_custom)
print("Accuracy:", accuracy_custom)
```

```
Dataset:
     Age  EstimatedSalary  Purchased
0    19            19000          0
1    35            20000          0
2    26            43000          0
3    27            57000          0
4    19            76000          0
5    27            58000          1
6    32           150000          1
7    25            33000          0
8    35            65000          1
9    26            80000          1
10   30            87000          1
11   40           120000          1
12   50            90000          1
13   48            60000          0
14   33            72000          1

--- In-built Model ---
Confusion Matrix:
 [[1 0]
 [0 2]]
Accuracy: 1.0

--- User Defined Model ---
F1 Score (Train): 0.7692307691692308
F1 Score (Test): 0.6666666665777777
Confusion Matrix:
 [[1 0]
 [1 1]]
Accuracy: 0.6666666666666666
```